ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| ассистент | | В. В. Жукалин |
|---|---|---|
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

# СОЗДАНИЕ СТРУКТУРЫ ВЕБ-ПРИЛОЖЕНИЯ С ПОМОЩЬЮ APP ROUTER. ПРОЕКТИРОВАНИЕ АВТОРИЗАЦИИ

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

| СТУДЕНТ гр. № 4321 | | И.А. Лаврешин |
|---|---|---|
| | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2026

## 1 Цель работы

Разработать структуру многостраничного веб-приложения SignalBoard с помощью App Router, настроить общую навигацию, создать динамический маршрут предметной сущности «сигнал» и реализовать клиентский прототип интерфейса входа и регистрации с явным пониманием различий между аутентификацией, управлением сессией и авторизацией.

## 2 Задание

Продолжить проект лабораторной работы № 1. Спроектировать набор разделов приложения. В каталоге app создать страницы и вложенные маршруты, задействовав общий корневой layout и навигацию на основе компонента Link. Добавить динамический маршрут для отдельного сигнала. Разделить Server Components и Client Components. Создать учебный контекст пользователя, формы входа и регистрации, разные состояния интерфейса для гостя и вошедшего пользователя. Объяснить ограничения клиентского прототипа. Проверить проект командами npm run lint и npm run build.

## 3 Ход выполнения задания

В рамках курса разрабатывается приложение SignalBoard – доска локальных сигналов для соседей.

Соответствие разделов каталога app и URL приведено в таблице. Корневой маршрут / через redirect() из next/navigation переводит пользователя на start. Лента сигналов размещена по адресу signals, карточка отдельного объекта – по адресу signals/[id]. Неизвестный идентификатор, например /signals/unknown, вызывает notFound().

| **Назначение** | **Каталог app** | **URL** |
|---|---|---|
| Редирект на старт | page.tsx | / → /start |
| Стартовая страница | start/page.tsx | /start |
| Лента сигналов | signals/page.tsx | /signals |
| Карточка сигнала | signals/[id]/page.tsx | /signals/{id} |
| О проекте | about/page.tsx | /about |
| Личный кабинет | cabinet/page.tsx | /cabinet |
| Вход | auth/login/page.tsx | /auth/login |
| Регистрация | auth/register/page.tsx | /auth/register |

Файл page.tsx задаёт содержимое сегмента URL. Файл layout.tsx задаёт общую оболочку: html, body, метаданные, DemoAuthProvider и компонент навигации AppNav. Layout не пересоздаётся полностью при переходах между вложенными страницами, поэтому состояние учебного пользователя сохраняется, пока открыта вкладка. Квадратные скобки в имени каталога [id] объявляют динамический сегмент. В актуальном App Router params приходят как Promise, поэтому идентификатор читается через await params.

Данные хранятся в типизированном массиве neighborSignals файла app/data/signals.ts. Это не база данных. У объекта есть id, title, summary и предметное поле category со значениями «ищу», «отдаю», «предупреждаю». Страницы start, signals, signals/[id] и about остаются Server Components. Директива «use client» стоит у AppNav, DemoAuthProvider, форм и кабинета, потому что им нужны useState, usePathname и обработчики событий.

```
app/
  favicon.ico
  globals.css
  layout.tsx
  page.tsx
  about/
    page.tsx
  auth/
    login/
      page.tsx
    register/
      page.tsx
  cabinet/
    page.tsx
  data/
    signals.ts
  providers/
    DemoAuthProvider.tsx
  signals/
    page.tsx
    [id]/
      page.tsx
  start/
    page.tsx
  ui/
    AppNav.tsx
    FeatureCard.tsx
```

Рисунок 1 – Структура каталога app

Компонент Link из next/link выполняет клиентские переходы без полной перезагрузки документа и сохраняет layout. Активный пункт навигации отмечается через usePathname и атрибут aria-current="page".

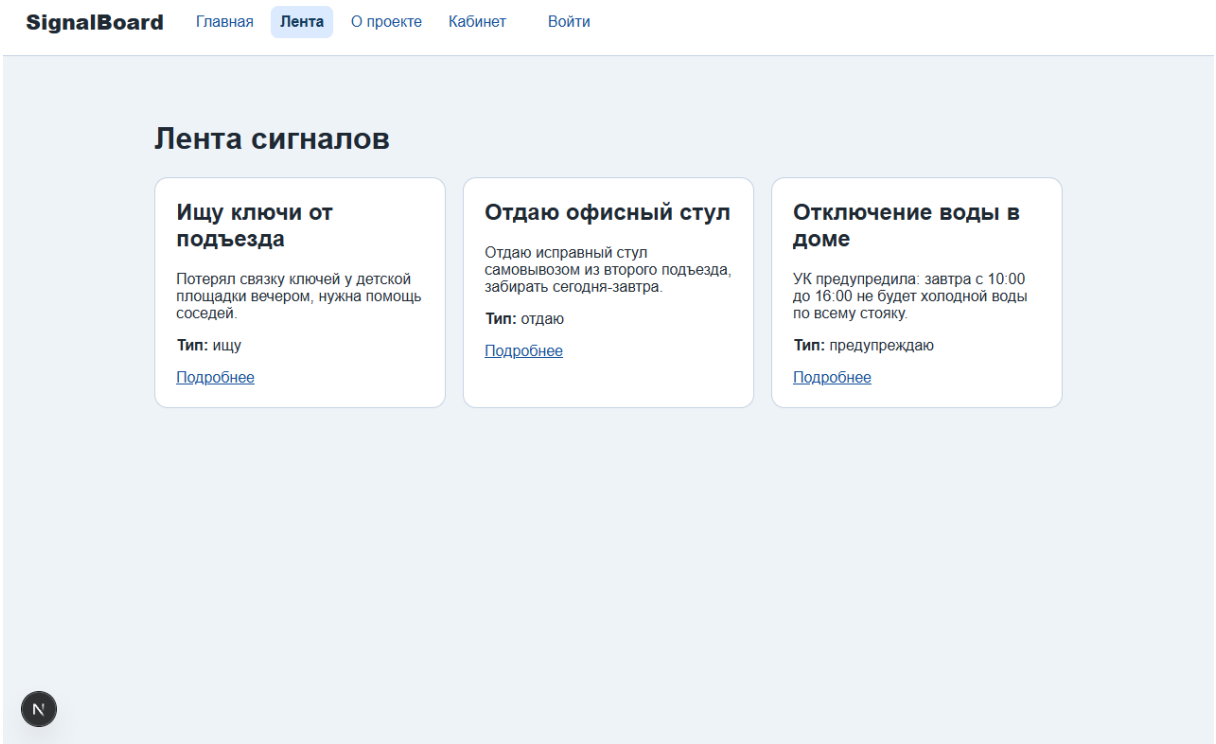



Рисунок 2 – Лента сигналов /signals

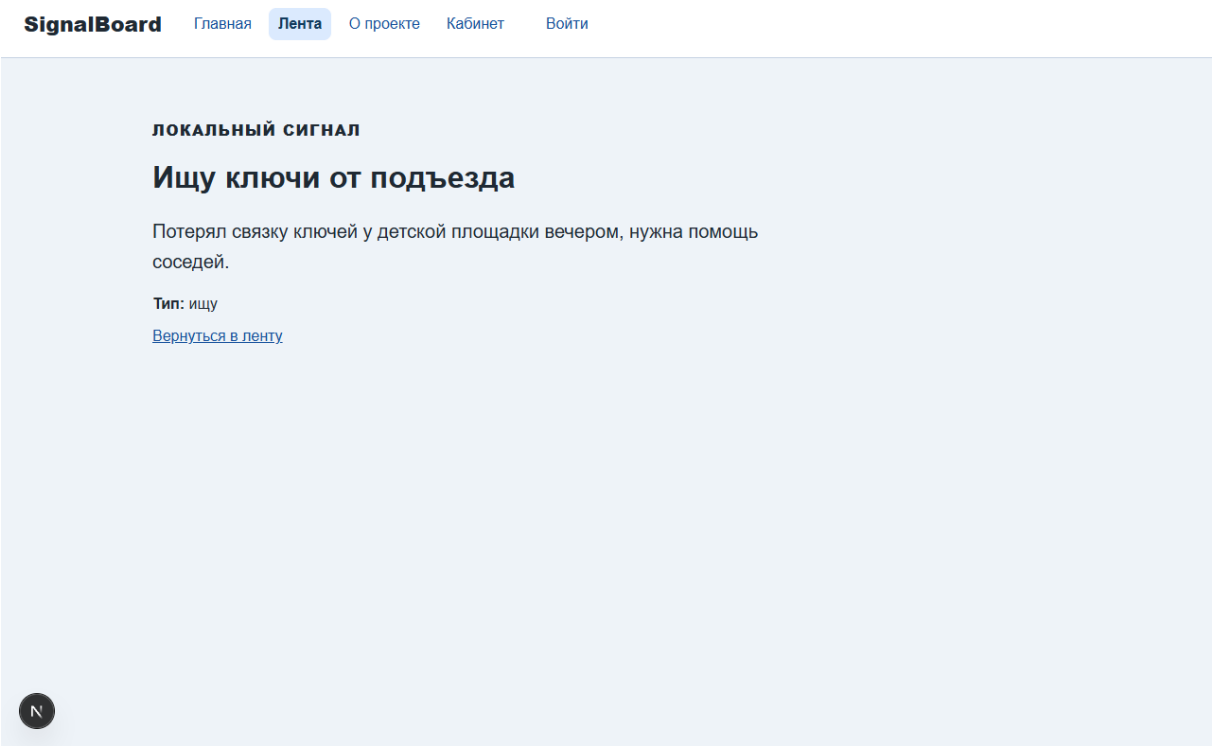



Рисунок 3 – Динамическая страница /signals/lost-keys

DemoAuthProvider хранит объект User | null в useState и отдаёт функции login, register и logout через React Context. Функция login берёт отображаемое имя из локальной части email, register – из поля имени. Пароль на клиенте не сохраняется. После обновления страницы состояние обнуляется, cookie-сессии нет, сервер пароль не проверяет. Это прототип интерфейса, а не средство безопасности: скрытие кнопки «Выйти» не защищает данные и будущие операции.

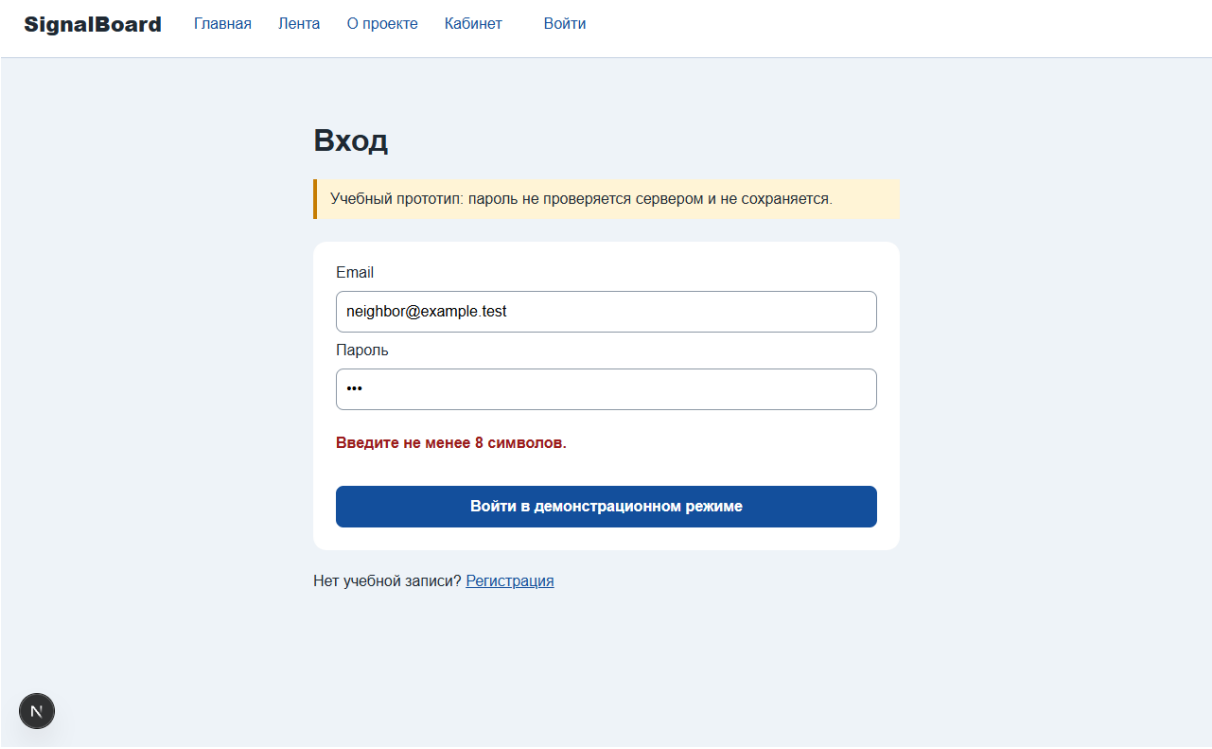



Рисунок 4 – Форма входа с ошибкой короткого пароля

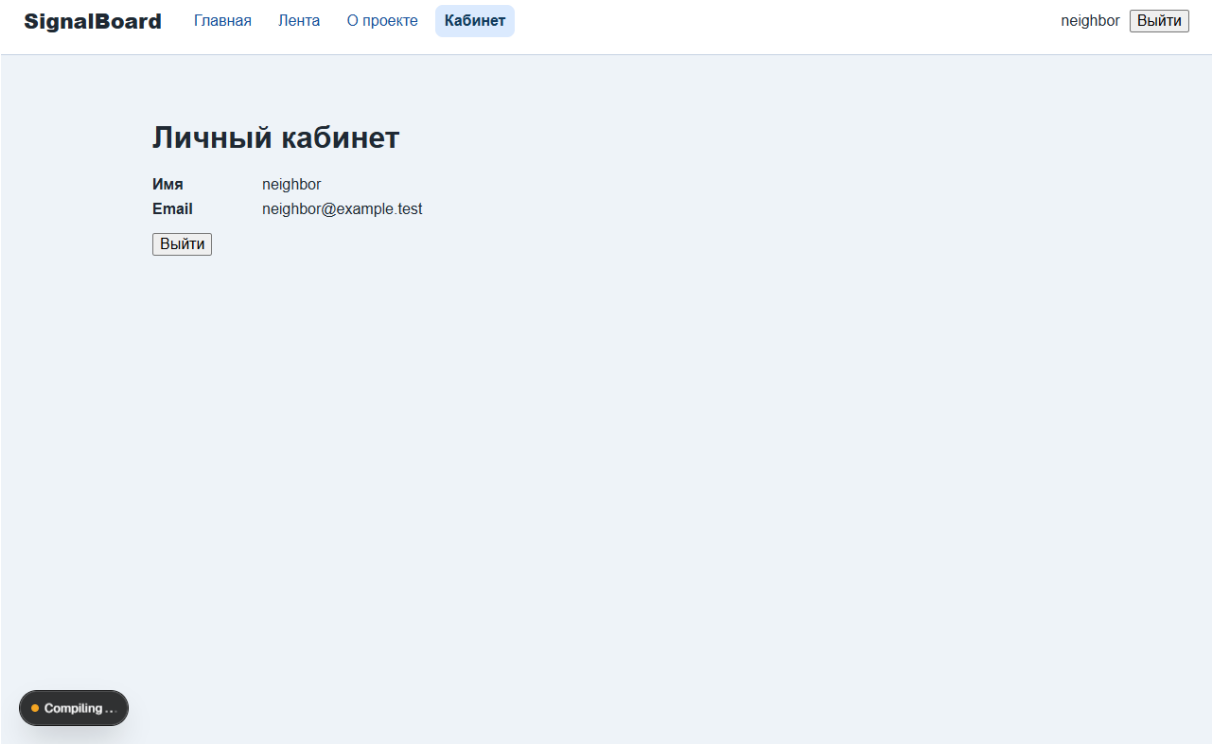



Рисунок 5 – Личный кабинет после учебного входа

Различия понятий, которые методичка требует явно зафиксировать, сведены в таблицу. Реализованный прототип закрывает только первый столбец на уровне интерфейса и не подменяет серверную сессию.

| **Понятие** | **Смысл** | **В данной работе** |
|---|---|---|

| | | |
|---|---|---|
| Аутентификация | Проверка, что пользователь тот, за кого себя выдаёт | Форма принимает email и пароль, пароль на сервер не уходит |
| Сессия | Сохранение факта входа между запросами | useState в провайдере; после reload пользователь исчезает |
| Авторизация | Проверка права выполнить операцию | Условный рендер кнопок; серверной проверки нет |

Выполнены проверки npm run lint и npm run build. ESLint завершился без ошибок. Production-сборка скомпилировалась успешно. Маршруты start, signals, about, cabinet, auth/login и auth/register сформированы как статические, маршрут signals/[id] – как динамический.

## 4 Вывод

В ходе работы каркас SignalBoard расширен до многостраничного приложения на App Router. Реализованы общая навигация, лента сигналов, динамический маршрут отдельного объекта, учебные формы входа и регистрации и условный интерфейс кабинета. Прототип явно отделён от защищённой авторизации: пароль не проверяется сервером, сессия не переживает обновление страницы, скрытие элементов UI не считается контролем доступа. Проверки lint и production-сборка завершились без ошибок.

## ПРИЛОЖЕНИЕ А

### Листинг с кодом программы

**Файл app/data/signals.ts**

```
// Описывает локальные сигналы соседей, демонстрационные данные и поиск объекта по id.

export type NeighborSignal = {
  id: string;
  title: string;
  summary: string;
  category: string;
};

export const neighborSignals: NeighborSignal[] = [
  {
    id: "lost-keys",
    title: "Ищу ключи от подъезда",
    summary: "Потерял связку ключей у детской площадки вечером, нужна помощь соседей.",
    category: "ищу",
  },
  {
    id: "give-chair",
    title: "Отдаю офисный стул",
    summary: "Отдаю исправный стул самовывозом из второго подъезда, забирать сегодня-завтра.",
    category: "отдаю",
  },
  {
    id: "water-outage",
    title: "Отключение воды в доме",
    summary: "УК предупредила: завтра с 10:00 до 16:00 не будет холодной воды по всему стояку.",
    category: "предупреждаю",
  },
];

export function getNeighborSignal(id: string): NeighborSignal | undefined {
  return neighborSignals.find((signal) => signal.id === id);
}
```

**Файл app/signals/page.tsx**

```
// Строит ленту карточек из массива локальных сигналов.

import Link from "next/link";
import { neighborSignals } from "../data/signals";

export default function SignalsPage() {
  return (
    <main>
      <h1>Лента сигналов</h1>
      <div className="card-grid">
        {neighborSignals.map((signal) => (
          <article className="card" key={signal.id}>
            <h2>{signal.title}</h2>
            <p>{signal.summary}</p>
            <p>
              <strong>Тип:</strong> {signal.category}
            </p>
            <Link href={`/signals/${signal.id}`}>Подробнее</Link>
          </article>
        ))}
      </div>
    </main>
  );
}
```

**Файл app/signals/[id]/page.tsx**

```
// Получает динамический id, находит объект и выводит страницу либо 404.

import Link from "next/link";
import { notFound } from "next/navigation";
import { getNeighborSignal } from "../../data/signals";

type SignalPageProps = {
  params: Promise<{ id: string }>;
};

export default async function SignalPage({ params }: SignalPageProps) {
  const { id } = await params;
  const signal = getNeighborSignal(id);

  if (!signal) {
    notFound();
  }

  return (
```

```
    <main>
      <p className="eyebrow">Локальный сигнал</p>
      <h1>{signal.title}</h1>
      <p className="lead">{signal.summary}</p>
      <p>
        <strong>Тип:</strong> {signal.category}
      </p>
      <Link href="/signals">Вернуться в ленту</Link>
    </main>
  );
}
```

**Файл app/providers/DemoAuthProvider.tsx**

// Хранит учебное состояние пользователя и предоставляет вход и выход через React Context.

```
"use client";

import {
  createContext,
  useContext,
  useMemo,
  useState,
  type ReactNode,
} from "react";

export type DemoUser = {
  name: string;
  email: string;
};

type DemoAuthContextValue = {
  user: DemoUser | null;
  login: (email: string) => void;
  register: (name: string, email: string) => void;
  logout: () => void;
};

const DemoAuthContext = createContext<DemoAuthContextValue | null>(null);

export default function DemoAuthProvider({
  children,
}: Readonly<{ children: ReactNode }>) {
```

```
  const [user, setUser] = useState<DemoUser | null>(null);

  const value = useMemo<DemoAuthContextValue>(
    () => ({
      user,
      login(email) {
        const name = email.split("@")[0]?.trim() || "Сосед";
        setUser({ name, email });
      },
      register(name, email) {
        setUser({ name: name.trim(), email });
      },
      logout() {
        setUser(null);
      },
    }),
    [user],
  );

  return (
    <DemoAuthContext.Provider value={value}>
      {children}
    </DemoAuthContext.Provider>
  );
}


export function useDemoAuth(): DemoAuthContextValue {
  const context = useContext(DemoAuthContext);

  if (!context) {
    throw new Error("useDemoAuth должен использоваться внутри
DemoAuthProvider");
  }

  return context;
}
```

**Файл app/auth/login/page.tsx**

```
// Обрабатывает учебный вход и переводит пользователя в личный кабинет.

"use client";
```

```
import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useDemoAuth } from "../../providers/DemoAuthProvider";

export default function LoginPage() {
  const router = useRouter();
  const { login } = useDemoAuth();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");

    if (password.length < 8) {
      setError("Введите не менее 8 символов.");
      return;
    }

    login(email.trim());
    router.push("/cabinet");
  }

  return (
    <main className="form-page">
      <h1>Вход</h1>
      <p className="demo-warning">
        Учебный прототип: пароль не проверяется сервером и не сохраняется.
      </p>

      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          value={email}
          onChange={(event) => setEmail(event.target.value)}
          autoComplete="email"
          required
        />

        <label htmlFor="password">Пароль</label>
```

```
        <input
          id="password"
          type="password"
          value={password}
          onChange={(event) => setPassword(event.target.value)}
          autoComplete="current-password"
          minLength={8}
          required
        />

        {error ? <p className="form-error">{error}</p> : null}

        <button type="submit">Войти в демонстрационном режиме</button>
      </form>

      <p>
        Нет учебной записи? <Link href="/auth/register">Регистрация</Link>
      </p>
    </main>
  );
}
```

**Файл app/auth/register/page.tsx**

```
// Создаёт учебного пользователя после отправки формы регистрации.

"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useDemoAuth } from "../../providers/DemoAuthProvider";

export default function RegisterPage() {
  const router = useRouter();
  const { register } = useDemoAuth();
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmation, setConfirmation] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
```

```
    setError("");

    if (password.length < 8) {
      setError("Пароль должен содержать не менее 8 символов.");
      return;
    }

    if (password !== confirmation) {
      setError("Введённые пароли не совпадают.");
      return;
    }

    register(name, email.trim());
    router.push("/cabinet");
  }

  return (
    <main className="form-page">
      <h1>Регистрация</h1>
      <p className="demo-warning">
        Данные существуют только в состоянии текущей вкладки и не
записываются в БД.
      </p>

      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="name">Имя</label>
        <input
          id="name"
          value={name}
          onChange={(event) => setName(event.target.value)}
          autoComplete="name"
          required
        />

        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          value={email}
          onChange={(event) => setEmail(event.target.value)}
          autoComplete="email"
          required
        />
```

```
        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          type="password"
          value={password}
          onChange={(event) => setPassword(event.target.value)}
          autoComplete="new-password"
          minLength={8}
          required
        />

        <label htmlFor="confirmation">Повторите пароль</label>
        <input
          id="confirmation"
          type="password"
          value={confirmation}
          onChange={(event) => setConfirmation(event.target.value)}
          autoComplete="new-password"
          minLength={8}
          required
        />

        {error ? <p className="form-error">{error}</p> : null}

        <button type="submit">Создать учебную запись</button>
      </form>

      <p>
        Уже есть учебная запись? <Link href="/auth/login">Войти</Link>
      </p>
    </main>
  );
}
```

**Файл app/layout.tsx**

// Подключает общие стили, навигацию и DemoAuthProvider ко всем страницам.

```
import type { Metadata } from "next";
import type { ReactNode } from "react";
import DemoAuthProvider from "./providers/DemoAuthProvider";
import AppNav from "./ui/AppNav";
import "./globals.css";
```

```
export const metadata: Metadata = {
  title: "SignalBoard",
  description: "Доска локальных сигналов соседям",
};

export default function RootLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  return (
    <html lang="ru">
      <body>
        <DemoAuthProvider>
          <AppNav />
          {children}
        </DemoAuthProvider>
      </body>
    </html>
  );
}
```

**Файл app/ui/AppNav.tsx**

```
// Формирует навигацию, отмечает активный маршрут и показывает состояние пользователя.

"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { useDemoAuth } from "../providers/DemoAuthProvider";

const links = [
  { href: "/start", label: "Главная" },
  { href: "/signals", label: "Лента" },
  { href: "/about", label: "О проекте" },
  { href: "/cabinet", label: "Кабинет" },
];

function isActive(pathname: string, href: string): boolean {
  return pathname === href || pathname.startsWith(`${href}/`);
}

export default function AppNav() {
```

```
  const pathname = usePathname();
  const { user, logout } = useDemoAuth();

  return (
    <header className="site-header">
      <Link className="brand" href="/start">
        SignalBoard
      </Link>

      <nav aria-label="Основная навигация">
        <ul className="nav-list">
          {links.map((link) => (
            <li key={link.href}>
              <Link
                className="nav-link"
                href={link.href}
                aria-current={isActive(pathname, link.href) ? "page" : undefined}
              >
                {link.label}
              </Link>
            </li>
          ))}
        </ul>
      </nav>

      {user ? (
        <div className="auth-summary">
          <span>{user.name}</span>
          <button type="button" onClick={logout}>
            Выйти
          </button>
        </div>
      ) : (
        <Link className="login-link" href="/auth/login">
          Войти
        </Link>
      )}
    </header>
  );
}
```

**Файл app/page.tsx**

```
// Перенаправляет корневой маршрут приложения на стартовую страницу.
```

```
import { redirect } from "next/navigation";

export default function HomePage() {
  // Выполняем серверное перенаправление на нужный маршрут.
  redirect("/start");
}
```

**Файл app/cabinet/page.tsx**

```
// Показывает гостевой экран либо сведения о вошедшем пользователе.

"use client";

import Link from "next/link";
import { useDemoAuth } from "../providers/DemoAuthProvider";

export default function CabinetPage() {
  const { user, logout } = useDemoAuth();
   if (!user) {
    return (
      <main>
        <h1>Личный кабинет</h1>
        <p>В демонстрационном состоянии пользователь отсутствует.</p>
        <Link href="/auth/login">Перейти ко входу</Link>
      </main>
    );
  }

  return (
    <main>
      <h1>Личный кабинет</h1>
      <dl className="profile-list">
        <dt>Имя</dt>
        <dd>{user.name}</dd>
        <dt>Email</dt>
        <dd>{user.email}</dd>
      </dl>
      <button type="button" onClick={logout}>
        Выйти
      </button>
    </main>
  );
}
```